DF 2026 가을 · 3주차 프로젝트 ① 분류

# 버섯 식용/독성 분류 과제 1~4단계 해설

범주형 인코딩 → X·y 추출 → train_test_split → fit·predict까지, 코드 한 줄씩과 결과 해석, 캔바 PPT 문구를 한 번에 정리했어요.

**116**
원-핫 후 특성 열 수
(21개 → 116개)

**6,499 / 1,625**
학습 / 테스트
(8 : 2, 독성 비율 48.2% 유지)

**1.000**
결정 트리 테스트 정확도
(오분류 0건)

**odor_n**
가장 중요한 특성
(중요도 0.62)

## 전체 흐름

**0. 준비**
파일 · 글꼴

**1. 인코딩**
정답 레이블
특성 원-핫

**2. X · y**
문제지와
정답지 분리

**3. 분리**
학습 80%
테스트 20%

**4. 예측**
fit → predict
→ 채점

## 함께 받은 파일

| 파일 | 용도 |
|---|---|
| `mushroom_steps_1_4.py` | 이 문서의 모든 코드가 들어 있는 파이썬 파일. `# %%` 표시로 덩어리가 나뉘어 있어요. |
| `mushrooms.csv` | 원본 데이터 (지난번에 받은 파일 그대로) |

## 코랩에서 실행하는 두 가지 방법

| 방법 | 하는 법 | 추천 상황 |
|---|---|---|
| **A. 덩어리별 실행** | .py 파일을 열어 `# %% [0]` ~ `[12]` 덩어리를 하나씩 코드 셀에 붙여 넣고 Shift+Enter | 처음 공부할 때 ✅ |
| **B. 한 번에 실행** | 왼쪽 📁에 .py와 csv를 올리고, 코드 셀에서 `%run mushroom_steps_1_4.py` | 결과만 빠르게 볼 때 |

csv를 미리 올리지 않았다면 [0] 덩어리가 업로드 창을 띄워 줘요. 그래프는 `figures` 폴더에 PNG로도 저장돼서 캔바에 바로 넣을 수 있어요.

# 0. 준비

라이브러리(남이 만들어 둔 도구 상자)를 불러오고, `mushrooms.csv`가 제자리에 있는지 확인해요. `sample_data` 폴더 안에 잘못 들어가 있으면 밖으로 꺼내고, 아예 없으면 업로드 창을 띄워요. 마지막으로 그래프 한글이 □□로 깨지지 않게 글꼴을 설정해요.

**[0] 준비: 라이브러리 · 파일 · 한글 글꼴**

```
import os, sys, subprocess
import pandas as pd                 # 표(DataFrame)를 다루는 도구
import numpy as np                  # 숫자 계산 도구
import matplotlib.pyplot as plt     # 그래프 도구
import seaborn as sns               # 통계 그래프 도구

# mushrooms.csv가 현재 폴더에 있는지 확인 → 없으면 sample_data 안을 찾고 → 그래도 없으면 업로드 창
if not os.path.exists('mushrooms.csv'):
    if os.path.exists('sample_data/mushrooms.csv'):
        os.rename('sample_data/mushrooms.csv', 'mushrooms.csv')     # 폴더 안에 잘못 들어간 파일을 밖으로
    else:
        try:
            from google.colab import files                          # 코랩 전용 업로드 도구
            print('[파일 선택] 버튼을 눌러 mushrooms.csv를 골라 주세요.')
            files.upload()
        except ImportError:
            sys.exit('mushrooms.csv를 이 파일과 같은 폴더에 넣어 주세요.')
print('mushrooms.csv 준비 완료:', os.path.exists('mushrooms.csv'))

# 그래프 한글 글꼴 (없으면 자동 설치)
try:
    import koreanize_matplotlib
except ImportError:
    subprocess.run([sys.executable, '-m', 'pip', 'install', '-q', 'koreanize-matplotlib'])
    try:
        import koreanize_matplotlib
    except ImportError:
        print('한글 글꼴 설치 실패: 그래프의 한글만 깨지고 나머지는 정상이에요.')
plt.rcParams['axes.unicode_minus'] = False

os.makedirs('figures', exist_ok=True)   # 그래프 이미지를 저장할 폴더 (PPT에 넣기 좋게)
```

# 1단계. 범주형 변수 인코딩

머신러닝 모델은 **숫자만 계산**할 수 있어요. 그런데 버섯 데이터는 전부 `e`, `p`, `a`, `f` 같은 **글자(범주형)**라서, 숫자로 바꾸는 **인코딩**이 필요해요.

### 두 가지 방법: 번호표 vs 체크박스 (냄새 열 예시)

| 원래 값 | 레이블 인코딩 (번호표) | 원-핫 인코딩 (체크박스) |
|---|---|---|
| a (아몬드) | 0 | odor_a=**1**, odor_f=0, odor_n=0 |
| f (불쾌한) | 1 | odor_a=0, odor_f=**1**, odor_n=0 |
| n (없음) | 2 | odor_a=0, odor_f=0, odor_n=**1** |
| **장점** | 열 수가 그대로 (간단) | 크기를 오해할 일이 없음 |
| **단점** | "n(2)이 a(0)보다 크다" 같은 **없는 크기 관계**를 모델이 믿을 수 있음 | 값 종류 수만큼 **열이 늘어남** |

### 팀의 선택 (이 코드도 똑같이 해요)

| 대상 | 방법 | 이유 |
|---|---|---|
| 정답 `class` | **레이블 인코딩** (e→0, p→1) | 정답은 열 하나여야 하고, 값이 2개뿐이라 크기 오해가 없음 |
| 특성 21개 | **원-핫 인코딩** | 냄새·색 같은 범주에는 크기가 없음. 로지스틱 회귀 등 다른 모델과도 공정하게 비교하려고 |

## 1-① 데이터 읽기 · 쓸모없는 열 지우기

**[1] 데이터 읽기 · veil-type 삭제 · ? 유지**

```
df = pd.read_csv('mushrooms.csv')            # 원본 (8124, 23)
print('원본:', df.shape)

if 'veil-type' in df.columns:                # 두 번 실행해도 에러 나지 않게 '남아 있을 때만' 삭제
    df = df.drop('veil-type', axis=1)        # 모든 버섯이 같은 값(p) → 구분에 쓸모없음
print('veil-type 삭제 후:', df.shape)        # (8124, 22)

# stalk-root의 '?'(결측)는 지우지 않고 하나의 범주로 둠 → '?'인 버섯의 71%가 독성이라 빈칸 자체가 단서
print(df['stalk-root'].value_counts())
```

**실행 결과**

```
원본: (8124, 23)
veil-type 삭제 후: (8124, 22)
stalk-root:  b 3776 / ? 2480 / e 1120 / c 556 / r 192
```

- **veil-type 삭제**: 모든 버섯이 같은 값(p)이라 식용·독성을 전혀 구분하지 못해요. 반 전원의 답이 같은 문제가 실력을 구분하지 못하는 것과 같아요.
- **'?' 2,480개 유지**: '?'인 버섯의 71%가 독성(전체 평균 48%)이라 **빈칸 자체가 단서**예요. 지우면 독버섯의 45%가 함께 사라져요.
- `if 'veil-type' in df.columns:` 는 셀을 두 번 실행해도 KeyError가 나지 않게 붙인 안전장치예요.

## 1-② 정답(class) → 레이블 인코딩

**[2] 정답 레이블 인코딩**

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()                                  # '글자 → 번호' 변환기
df['class'] = encoder.fit_transform(df['class'])          # fit(값 종류 배우기) + transform(번호로 바꾸기)
mapping = {label: num for num, label in enumerate(encoder.classes_)}
print('변환 규칙:', mapping)                              # {'e': 0, 'p': 1} → 0 = 식용, 1 = 독성
print(df['class'].value_counts())                         # 0: 4208, 1: 3916
```

**실행 결과**

```
변환 규칙: {'e': 0, 'p': 1}
class:  0 → 4208개 (식용) / 1 → 3916개 (독성)
```

**fit과 transform: 사이킷런 전체에서 반복되는 패턴**

`fit()` 은 데이터를 보고 **규칙을 배우고**("e는 0번, p는 1번"), `transform()` 은 그 규칙을 **적용**해요. `fit_transform()` 은 둘을 한 번에 해요. 4단계의 `fit()` (학습) → `predict()` (예측)도 같은 패턴이에요.

> **0 = 식용, 1 = 독성 — 꼭 기억하기**
>
> 4단계에서 재현율·혼동행렬을 읽을 때 기준이 돼요. 팀 PPT 1번 슬라이드에 `(e : 1, p : 0)`으로 반대로 적힌 곳이 있으니 `(e : 0, p : 1)`로 고쳐야 해요.

### 1-③ 특성 21개 → 원-핫 인코딩 (메인)

**[3] 특성 원-핫 인코딩**

```
X_onehot = pd.get_dummies(df.drop('class', axis=1), dtype=int)
# df.drop('class', axis=1) : 정답을 뺀 특성 21개
# pd.get_dummies(...)      : 값마다 0/1 열을 만듦 (원-핫)
# dtype=int                : True/False 대신 1/0
print('원-핫 전:', df.drop('class', axis=1).shape)       # (8124, 21)
print('원-핫 후:', X_onehot.shape)                       # (8124, 116)

odor_cols = [c for c in X_onehot.columns if c.startswith('odor_')]
print(X_onehot[odor_cols].head(3))                       # 행마다 자기 냄새 칸 하나만 1
```

**실행 결과**

```
원-핫 전: (8124, 21)
원-핫 후: (8124, 116)
   odor_a odor_c odor_f odor_l odor_m odor_n odor_p odor_s odor_y
0       0      0      0      0      0      0      1      0      0   ← 0번 버섯 냄새 = p
1       1      0      0      0      0      0      0      0      0   ← 1번 버섯 냄새 = a
2       0      0      0      1      0      0      0      0      0   ← 2번 버섯 냄새 = l
```

21개 열의 값 종류를 모두 더하면 116이라 열이 **21 → 116개**로 늘어나요. 행마다 **딱 하나(one)만 1로 켜져(hot)** 있어서 '원-핫'이라고 불러요.

### 1-④ (비교용) 특성 21개 → 레이블 인코딩

과제 1단계에 두 방법이 모두 적혀 있어서, 비교용으로 레이블 인코딩 버전(`X_label`)도 만들어 둬요. 4단계에서 정확도를 비교해요.

**[4] 비교용 레이블 인코딩**

```
X_label = df.drop('class', axis=1).apply(lambda col: LabelEncoder().fit_transform(col))
# .apply(함수): 열마다 같은 함수를 적용 / 각 열의 글자를 알파벳 순서대로 0, 1, 2 … 로
print('레이블 인코딩 후:', X_label.shape)                # (8124, 21) → 열 수 그대로
```

**실행 결과**

```
레이블 인코딩 후: (8124, 21)   ← 열 수는 그대로, 칸마다 0~11 사이 번호
```

- `.apply(함수)` : 표의 **열마다** 같은 함수를 적용해요.
- `lambda col: ...` : 이름 없는 짧은 함수예요. `col`이 열 하나를 뜻해요.

## 2단계. X, y를 새 DataFrame으로 추출

머신러닝에서는 **문제(X)와 정답(y)을 따로** 모델에 넘겨요. X는 **시험지 지문**, y는 **정답지**예요.

### [5] X, y 추출

```
X = X_onehot.copy()          # 특성(문제지): 원-핫 116열 DataFrame. copy()로 원본 보호
y = df['class']              # 정답(정답지): 0/1 한 열 (Series)

print('X:', type(X).__name__, X.shape)                  # DataFrame (8124, 116)
print('y:', type(y).__name__, y.shape)                  # Series (8124,)
print('행 짝이 맞는가?', (X.index == y.index).all())     # True
print(y.to_frame().head(3))                             # y를 DataFrame 모양으로도 확인
```

**실행 결과**

```
X: DataFrame (8124, 116)
y: Series (8124,)
행 짝이 맞는가? True
```

- **대문자 X / 소문자 y**: X는 여러 열의 표(DataFrame), y는 열 하나(Series)라는 관례적인 표기예요.
- `.copy()`: 복사본을 만들어서 X를 고쳐도 원본 `X_onehot`은 안전해요.
- 과제 문구가 "새 **DataFrame**"이라, y도 `y.to_frame()`으로 표 모양을 확인했어요. 모델에 넣을 때는 Series 그대로가 편해요.
- **행 짝 확인**: X와 y의 행 번호가 같아야 각 버섯과 정답이 제자리에 짝지어져요.

# 3단계. train_test_split으로 데이터 분리

모델이 **공부한 문제로 시험을 보면** 점수가 높아도 진짜 실력인지 알 수 없어요. 그래서 데이터를 둘로 나눠요.

| 이름 | 비율 | 역할 | 비유 |
|---|---|---|---|
| **학습 데이터 (train)** | 80% | 모델이 공부(fit)하는 데이터 | 문제집 |
| **테스트 데이터 (test)** | 20% | 공부가 끝난 뒤 실력을 재는 데이터 | **처음 보는** 시험지 |

**[6] 학습/테스트 분리**

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # 테스트 20% (1,625개)
    random_state=42,     # 누가 몇 번 실행해도 같은 결과
    stratify=y           # 학습·테스트 모두 식용:독성 비율을 원본과 같게
)
print('X_train:', X_train.shape, '| X_test:', X_test.shape)   # (6499, 116) | (1625, 116)
print(f'독성 비율  전체 {y.mean():.2%} / 학습 {y_train.mean():.2%} / 테스트 {y_test.mean():.2%}')
print('테스트 정답 개수:', y_test.value_counts().to_dict())   # {0: 842, 1: 783}
```

**실행 결과**

```
X_train: (6499, 116) | X_test: (1625, 116)
독성 비율  전체 48.20% / 학습 48.21% / 테스트 48.18%
테스트 정답 개수: {0: 842, 1: 783}
```

| 파라미터 | 값 | 뜻과 선택 이유 |
|---|---|---|
| `test_size` | 0.2 | 테스트에 20%. 8,124개 중 1,625개라 시험지로 충분해요. |
| `stratify` | `y` | 학습·테스트 모두 **식용:독성 비율을 원본과 같게** 맞춰요(층화 추출). 결과에서 세 비율이 모두 48.2%로 같아요. |
| `random_state` | 42 | 무작위로 섞는 방식을 고정해서, 조원 누구나 같은 숫자를 재현할 수 있게 해요. 42는 관례적으로 쓰는 숫자예요. |

결과는 순서대로 **학습 문제, 테스트 문제, 학습 정답, 테스트 정답** 4개로 나와요. 이 순서를 바꿔 쓰면 엉뚱한 결과가 나오니 주의하세요.

# 4단계. fit() · predict()로 예측

### 모델 선택: 결정 트리를 메인으로

| 모델 | 원리 | 비유 |
|---|---|---|
| **결정 트리** ✅ 메인 | 질문을 이어가며 데이터를 나눔 | **스무고개** ("냄새가 없나요?" → "포자가 녹색인가요?") |
| 랜덤 포레스트 (비교) | 조금씩 다른 결정 트리 100개의 다수결 | 스무고개 선수 100명의 투표 |

- EDA에서 본 "냄새 → 포자 색" 구조가 **스무고개 방식과 딱 맞아요.**

- 어떤 질문으로 나눴는지 **그림으로 보여줄 수 있어서** 발표에 좋아요.
- 6단계에서 `max_depth`(질문 횟수 제한)에 따라 성능이 확실히 달라져서 튜닝 이야깃거리가 생겨요.
- **스케일링은 하지 않아요.** 트리는 "이 값이 0.5보다 큰가?"처럼 기준으로 나누기만 해서 숫자 크기를 맞출 필요가 없고, 원-핫 값은 이미 0 아니면 1이에요.

## 4-① 학습(fit) → 예측(predict) → 채점

**[7] 결정 트리 학습 · 예측 · 채점**

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, recall_score

dt = DecisionTreeClassifier(random_state=42)   # 모델 만들기 (기본 설정)
dt.fit(X_train, y_train)                       # 학습: 문제집 + 정답으로 규칙을 배움
pred = dt.predict(X_test)                      # 예측: 처음 보는 시험지의 정답을 맞힘

print(pd.DataFrame({'실제': y_test.values[:10], '예측': pred[:10]}).T)   # 앞 10개 비교
print('테스트 정확도:', accuracy_score(y_test, pred))
print('학습 정확도  :', accuracy_score(y_train, dt.predict(X_train)))
print('독성 재현율  :', recall_score(y_test, pred))   # 실제 독버섯 중 '독성'으로 찾아낸 비율
```

실행 결과

```
     0  1  2  3  4  5  6  7  8  9
실제  1  1  0  1  1  0  0  0  0  1
예측  1  1  0  1  1  0  0  0  0  1
테스트 정확도: 1.0
학습 정확도  : 1.0
독성 재현율  : 1.0
```

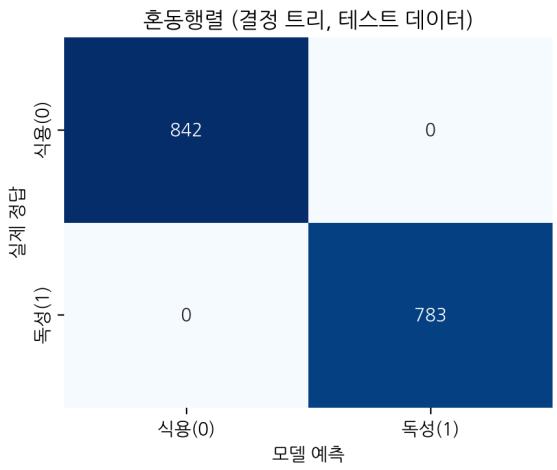
| 지표 | 뜻 | 결과 |
|---|---|---|
| **정확도** | 전체 중 맞힌 비율 | 1.000 |
| **독성 재현율** | 실제 독버섯 중 "독성"이라고 **찾아낸** 비율. 독버섯을 식용이라 하는 실수가 가장 위험해서 따로 봐요. | 1.000 |

## 4-② 혼동행렬: 무엇을 맞히고 무엇을 틀렸나

**[8] 혼동행렬 그래프**

```
cm = confusion_matrix(y_test, pred)            # [[식용→식용, 식용→독성], [독성→식용, 독성→독성]]
print(cm)

fig, ax = plt.subplots(figsize=(4.5, 3.8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax,
            xticklabels=['식용(0)', '독성(1)'], yticklabels=['식용(0)', '독성(1)'])
ax.set_xlabel('모델 예측'); ax.set_ylabel('실제 정답')
ax.set_title('혼동행렬 (결정 트리, 테스트 데이터)')
plt.tight_layout(); plt.savefig('figures/confusion_matrix.png', dpi=200); plt.show()
```




| | 예측 식용 | 예측 독성 |
|---|---|---|
| **실제 식용** | ✅ 맞힘 (842) | ⚠️ 아깝지만 안전 (0) |
| **실제 독성** | ☠️ **가장 위험** (0) | ✅ 맞힘 (783) |

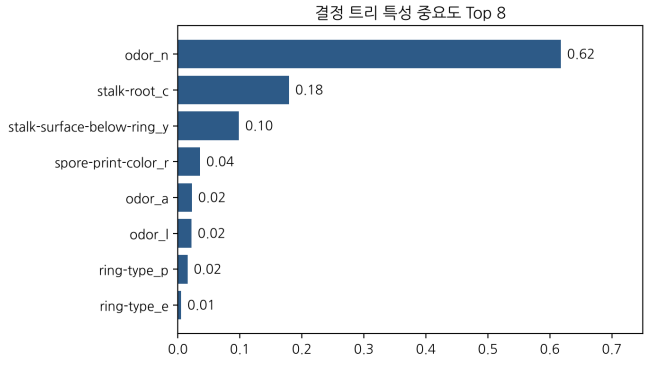
테스트 1,625개를 **하나도 틀리지 않았어요.** 가장 위험한 칸(독성 → 식용)도 0이에요.

## 4-③ 모델이 중요하게 본 특성

**[9] 특성 중요도 그래프**

```
imp = pd.Series(dt.feature_importances_, index=X.columns).sort_values(ascending=False)
print(imp.head(5).round(3))                    # 모두 더하면 1

top = imp.head(8)[::-1]                        # 상위 8개 (큰 값이 위로 오게 뒤집음)
fig, ax = plt.subplots(figsize=(7, 4))
ax.barh(top.index, top.values, color='#2E5C8A')
for i, v in enumerate(top.values):
    ax.text(v + 0.01, i, f'{v:.2f}', va='center')
ax.set_xlim(0, 0.75); ax.set_title('결정 트리 특성 중요도 Top 8')
plt.tight_layout(); plt.savefig('figures/feature_importance.png', dpi=200); plt.show()
```



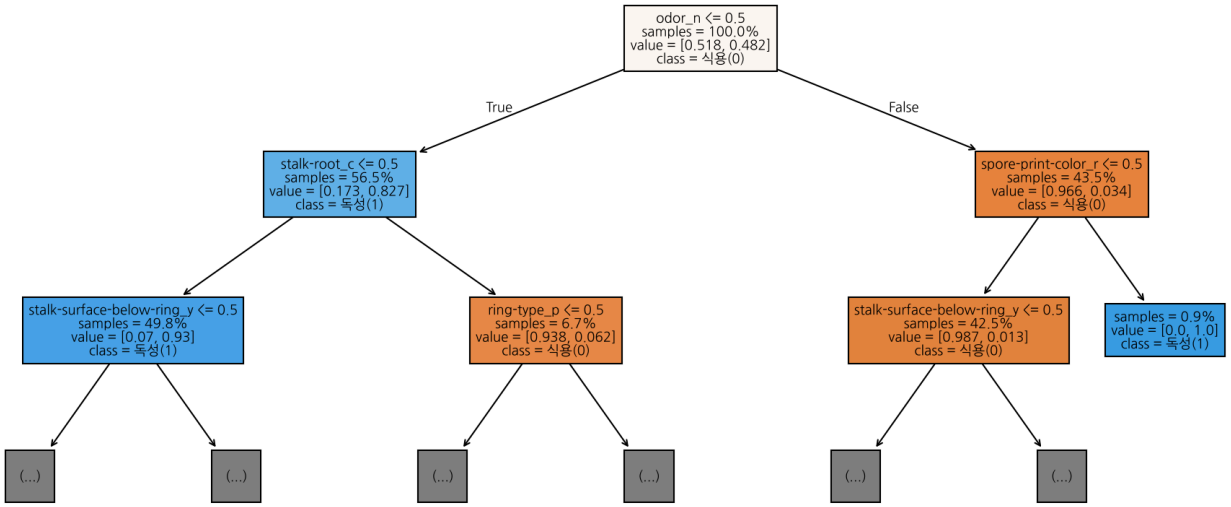

- **odor_n(냄새 없음 여부) 0.62**로 압도적이에요. EDA의 "냄새 하나로 대부분 갈린다"와 일치해요.
- stalk-root_c, stalk-surface-below-ring_y, spore-print-color_r(녹색 포자)은 **냄새 없는 버섯들을 마저 가르는 데** 쓰였어요.

## 4-④ 결정 트리 그림 (스무고개 확인)

**[10] 결정 트리 그림 (위쪽 3단계)**

```
from sklearn.tree import plot_tree

fig, ax = plt.subplots(figsize=(16, 7))
plot_tree(dt, max_depth=2, feature_names=X.columns, class_names=['식용(0)', '독성(1)'],
          filled=True, impurity=False, proportion=True, fontsize=9, ax=ax)
plt.savefig('figures/decision_tree.png', dpi=150, bbox_inches='tight'); plt.show()
print('트리 깊이:', dt.get_depth(), '/ 잎 개수:', dt.get_n_leaves())
```




트리 전체 깊이 7, 마지막 칸(잎) 14개 · 위쪽 3단계만 표시

- 네모 하나 = **질문 하나**. 맨 위는 `odor_n <= 0.5`, 즉 "냄새 없음이 **아닌가**?"예요. 원-핫 값은 0/1이라 `<= 0.5`는 "0인가?"와 같아요.
- 왼쪽 화살표 = **예(True)**, 오른쪽 = **아니오(False)**. `value = [식용 비율, 독성 비율]`, 주황 = 식용 쪽, 파랑 = 독성 쪽이에요.
- **오른쪽(냄새 없음, 43.5%)**: 97%가 식용 → 다음 질문 "포자가 녹색인가?" → 녹색이면 **바로 독성 100%**.
- **왼쪽(냄새 있음, 56.5%)**: 83%가 독성 → 다른 특성으로 나머지를 마저 가려요.
- EDA에서 손으로 해본 **"냄새 → 포자 색" 스무고개를 모델이 스스로 찾아냈어요.**

### 4-⑤ 비교: 랜덤 포레스트, 레이블 인코딩 버전

**[11] 비교 실험**

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42).fit(X_train, y_train)        # 트리 100개의 다수결
rf_acc = accuracy_score(y_test, rf.predict(X_test))

XL_train, XL_test, yL_train, yL_test = train_test_split(                  # 레이블 인코딩 특성도 같은 조건으로 분리
    X_label, y, test_size=0.2, random_state=42, stratify=y)
dt_label = DecisionTreeClassifier(random_state=42).fit(XL_train, yL_train)
label_acc = accuracy_score(yL_test, dt_label.predict(XL_test))

print(pd.DataFrame({'특성 인코딩': ['원-핫(116열)', '원-핫(116열)', '레이블(21열)'],
                    '모델': ['결정 트리', '랜덤 포레스트', '결정 트리'],
                    '테스트 정확도': [accuracy_score(y_test, pred), rf_acc, label_acc]}))
```

| 특성 인코딩 | 모델 | 테스트 정확도 |
|---|---|---|
| 원-핫 (116열) | 결정 트리 ✅ | 1.000 |
| 원-핫 (116열) | 랜덤 포레스트 | 1.000 |
| 레이블 (21열) | 결정 트리 | 1.000 |

- 결정 트리는 기준값으로 나누기만 해서 **레이블 인코딩이어도 잘 작동해요.** 그래도 팀은 다른 모델과 공정하게 비교하려고 원-핫을 메인으로 써요.

- 랜덤 포레스트도 1.000이라 더 나을 게 없어요 → **설명하기 쉬운 결정 트리 한 개**로 충분해요.

### 4-⑥ (보너스) 질문 횟수 제한 · 냄새 뺀 실험

**[12] max_depth별 성능 · 냄새 제외 실험**

```
rows = []
for depth in [1, 2, 3, 4, 5, None]:                    # None = 제한 없음(기본값)
    m = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(X_train, y_train)
    p = m.predict(X_test)
    rows.append({'max_depth': '제한 없음' if depth is None else depth,
                 '정확도': round(accuracy_score(y_test, p), 4),
                 '독성 재현율': round(recall_score(y_test, p), 4),
                 '놓친 독버섯': int(confusion_matrix(y_test, p)[1, 0])})
print(pd.DataFrame(rows))

no_odor = [c for c in X.columns if not c.startswith('odor_')]                # odor 9개 열 제외
dt_no_odor = DecisionTreeClassifier(random_state=42).fit(X_train[no_odor], y_train)
print('냄새 없이 정확도:', accuracy_score(y_test, dt_no_odor.predict(X_test[no_odor])))
```

| max_depth | 정확도 | 독성 재현율 | 놓친 독버섯 |
|---|---|---|---|
| 1 | 0.8855 | 0.9706 | 23 |
| 2 | 0.9489 | 0.9732 | 21 |
| 3 | 0.9822 | 0.9962 | 3 |
| 4 | 0.9914 | 0.9974 | 2 |
| 5 | 0.9988 | 0.9974 | 2 |
| 제한 없음 | 1.0000 | 1.0000 | 0 |

`max_depth` = 스무고개에서 질문을 **최대 몇 번**까지 할지 정하는 설정값(하이퍼파라미터)이에요. 질문 1번(냄새만)으로도 88.6%, 늘릴수록 올라가요. **6단계 GridSearchCV**에서 이런 값을 자동으로 비교해요.

**냄새(odor 9개 열)를 빼도 정확도 1.000**이에요. 포자 색·주름·줄기 표면 같은 다른 특성만으로도 충분히 구분돼요.

**정확도 100%, 믿어도 될까요? (발표 대비)**

① **처음 보는 테스트 데이터**에서 나온 점수예요. ② EDA에서 냄새 하나로 9종 중 8종이 0% 또는 100%로 갈리는 **원래 쉽게 구분되는 데이터**임을 확인했어요. ③ 우연히 시험지가 쉬웠던 건 아닌지 **5단계 교차 검증**으로 한 번 더 확인할 거예요.

## 캔바 PPT에 옮길 내용

조별 PPT 구성 예시의 **1번(전처리 요약)**과 **2번(분리 및 모델)**이 1~4단계 범위예요. 아래 문구를 그대로 옮기고, 그림은 `figures` 폴더의 PNG를 쓰면 돼요.

### 슬라이드 A. 전처리 요약: 인코딩 · 스케일링

- **삭제**: veil-type (모든 값이 같아 정보 없음) → 행은 하나도 삭제하지 않음
- **결측(?) 처리**: stalk-root의 ? 2,480개를 하나의 범주로 유지 (?인 버섯의 71%가 독성)
- **정답 인코딩**: class → LabelEncoder (e=0 식용, p=1 독성)
- **특성 인코딩**: 21개 → One-Hot (get_dummies) → **116개 열**
  이유: 범주에 크기·순서가 없음, 다른 모델과 공정하게 비교 / 비교: 레이블 인코딩(21열)도 결정 트리 정확도 동일(1.000)
- **스케일링**: 미적용 → 트리 모델은 값의 크기에 영향받지 않고, 원-핫 값은 이미 0/1

### 슬라이드 B. 분리 및 모델

- **분리**: train_test_split(test_size=0.2, stratify=y, random_state=42)
  학습 6,499개 / 테스트 1,625개, 독성 비율 모두 48.2%로 유지
- **Estimator**: DecisionTreeClassifier(random_state=42), 기본 설정
  선택 이유: 스무고개 방식이 데이터 구조와 맞고, 판단 과정을 그림으로 설명 가능
- **결과**: 테스트 정확도 **1.000**, 독성 재현율 **1.000** (오분류 0건)
- **해석**: 가장 중요한 특성은 odor_n(냄새 없음, 중요도 0.62). 냄새를 빼도 정확도 1.000
- 📸 그림 추천: 혼동행렬, 특성 중요도, 트리 그림

### 점수 비교표 (분류) — 5·6단계 후 채우기

| 지표 | 기본 모델 | 교차 검증 평균 | 튜닝 후 |
|---|---|---|---|
| 정확도 | **1.000** | (5단계) | (6단계) |

### 예상 질문 & 답

| 질문 | 답 |
|---|---|
| 정확도 100%면 과적합 아닌가요? | 처음 보는 테스트 데이터 점수이고, EDA에서 냄새만으로 대부분 갈리는 데이터임을 확인했어요. 5단계 교차 검증으로 한 번 더 확인할 예정이에요. |
| 트리 모델인데 왜 원-핫을 했나요? | 트리는 레이블 인코딩도 되지만(실제로 정확도 동일), 로지스틱 회귀 등 숫자 크기를 쓰는 모델과 같은 데이터로 공정하게 비교하려고 원-핫으로 통일했어요. |
| 왜 stratify를 썼나요? | 학습과 테스트의 식용:독성 비율을 원본(48.2%)과 같게 맞춰서, 한쪽에 독버섯이 몰리지 않게 하려고요. |
| random_state는 왜 넣었나요? | 실행할 때마다 같은 결과가 나오게 해서 조원 누구나 같은 숫자를 재현할 수 있게 하려고요. |
| 왜 랜덤 포레스트 대신 결정 트리인가요? | 둘 다 1.000이라 성능 차이가 없고, 결정 트리는 판단 과정을 그림으로 설명할 수 있어서요. |

## 자주 나는 에러

| 에러 | 원인 | 해결 |
|---|---|---|
| `FileNotFoundError` | csv가 없거나, 런타임이 끊겨 업로드한 파일이 사라짐 | [0] 덩어리를 다시 실행 (업로드 창이 뜸) |
| `NameError: name 'X_train' ...` | 위 덩어리를 건너뛰고 아래부터 실행 | [0]부터 순서대로, 또는 런타임 → 모두 실행 |
| 그래프 한글이 □□ | 글꼴 미설정 | [0] 덩어리 실행 |
| 결과 숫자가 이 문서와 다름 | `random_state`를 빼거나 바꿈 | `random_state=42` 그대로 두기 |
| 에러가 줄줄이 수십 개 | 앞 덩어리 하나가 실패해서 뒤가 따라 실패 | **첫 번째 에러만** 해결 → 런타임 → 세션 다시 시작 → 모두 실행 |

### 다음 할 일

- **5단계** 교차 검증 (KFold, Stratified KFold) → 폴드별 점수와 평균
- **6단계** GridSearchCV → max_depth 등 최적 파라미터, 튜닝 전후 점수
- **7단계** 파라미터 선택 이유를 PPT에 작성 → 노션 A반 과제 3주차에 업로드